TYPES OF GRAPHS

GROUP 2:

Rena Fathima
Malavika S
Jennifer Rose
Parvana
Premoj
Anagha
Krishnapriya S

Introduction To Graphs:

A **Graph** is a **non-linear data structure** made up of **vertices (nodes)** and **edges (links)**.
It is used to represent **relationships** between objects.
Graphs are widely used in **social networks, maps, and computer networks**.
They can be **directed** or **undirected**, depending on whether the connections have direction.

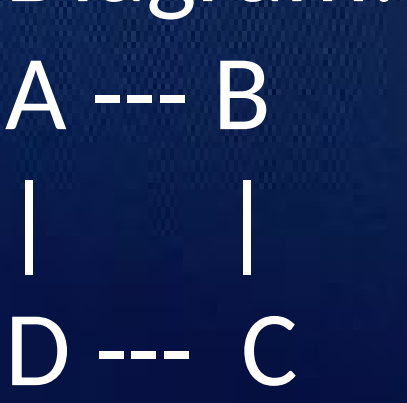
Types of Graphs:

Finite Graph:

A finite graph has a limited number of vertices and edges.
Commonly used in computer applications.
Easy to represent using adjacency matrix or list.

Example:
Vertices = {A, B, C, D}
Edges = {(A,B), (B,C), (C,D), (D,A)}

Diagram:



Infinite Graph:

An infinite graph has an unlimited number of vertices or edges.
Used mostly in theoretical or mathematical models.
Not practical for computer implementation.
Example:
Integers connected in a line:
— (-2) — (-1) — 0 — 1 — 2 — …

Simple Graph:

A simple graph has:

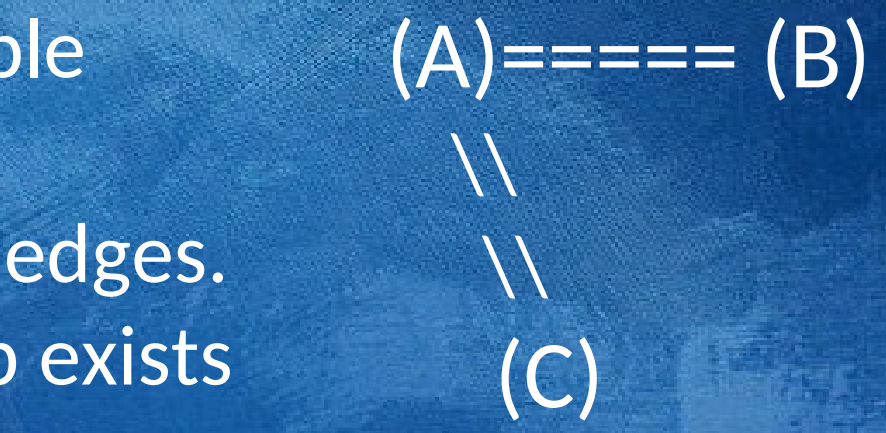
No loops (no edge connects a vertex to itself)
No multiple edges between the same pair of vertices.
Every edge joins two different vertices exactly once.
Example Diagram:
A — B — C

🔔 Conclusion:
Finite graphs → Used in real-world problems.
Infinite graphs → Mainly theoretical.
Simple graphs → Basis for most graph algorithms (DFS, BFS, etc.)

Multi Graph:

A multigraph is a graph that allows multiple edges between the same pair of vertices.

- These multiple edges are called parallel edges.
- Useful when more than one relationship exists between two nodes.



Weighted Graph:

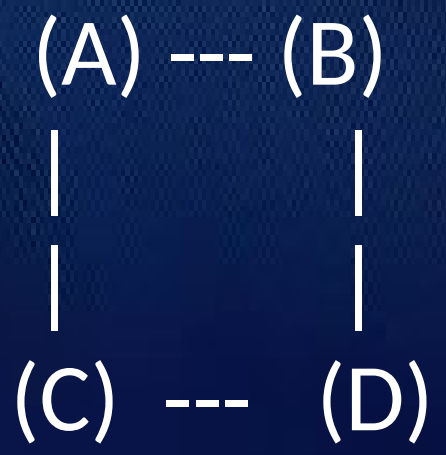
Edges in a weighted graph carry a value or weight.

- Weights can represent cost, distance, time, or capacity.
- Stored in C using adjacency lists or matrices with weights.



Unweighted Graph:

- In an unweighted graph, all edges are equal — no special value or cost.
- Representation: adjacency matrix or adjacency list.
- Used when only connections matter, not distances.



DIRECTED GRAPHS:

A Directed Graph, also called a Digraph, is a graph in which every edge has a direction. This means the relationship between two vertices goes one way only, and is represented using an arrow (→).

For example, if there is an edge from A to B, it means you can go from A to B, but not necessarily from B to A.

Main Concepts in Directed Graph:

Directed Edges:
Each edge has a direction showing a **one-way** connection between two vertices

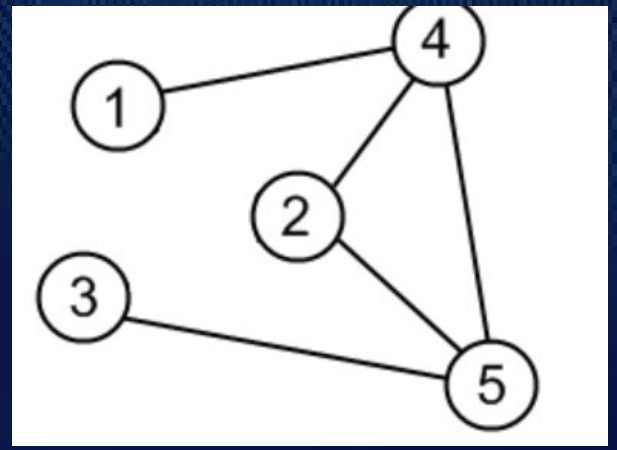
Indegree and Outdegree:
•Indegree: Number of incoming edges to a vertex.
•Outdegree: Number of outgoing edges from a vertex.

Cycles:
A cycle occurs when a path starts and ends at the same vertex following the direction of edges.

Paths and Reachability:
Paths must follow the direction of edges, and are used to determine if one vertex can be reached from another.

Undirected graphs:

An **Undirected Graph** is a graph in which **edges have no direction**.
This means the relationship between two vertices is **two-way** or **mutual**.
If there is an edge between **A and B**, you can travel **both from A to B and from B to A** without direction arrows.



Main Concepts in Undirected Graph:

•**Two-Way Relationship:**
The connection works **both ways**.
Example: If a road connects two houses, you can travel both directions on it.
•**Degree of a Vertex:**
The **degree** is just **how many lines** (edges) are touching the node.
(No separate in/out counts.)
•**No Direction Needed in Travel:**
When moving through the graph, you don't have to worry about directions —
you can go back and forth freely.
•**Common Real-Life Example:**
Social friendships, **undirected road maps**, **network of connected computers** in LAN.
•**Connectedness:**
If you can start at one vertex and **eventually reach every other vertex**, the graph is called **connected**

Number Of Edges In An Undirected Graph:

To calculate the total number of unique edges (|E|) in an undirected graph.

Based on the Handshaking Lemma principle:

$$|E| = \frac{1}{2} \sum_{v \in V} \deg(v)$$

Sum of degrees of all vertices = 2 × Number of edgesEach edge contributes 1 to the degree of two vertices (its endpoints).

$$|E| = \frac{\text{Sum of all vertex degrees}}{2}$$

Algorithm Steps:

Step 1: Initialize edgeCount = 0
Step 2: For each vertex v in the graph→ Add its degree to edgeCount
Step 3: Divide the total by 2 to remove double-counting
Step 4: Return edgeCount / 2 as the number of edges

C Program Implementation

```
#include <stdio.h>
int count_undirected_edges(int V, const int degrees[])
{
  int sum = 0;
    for (int i = 0; i < V; i++)
    sum += degrees[i];
    return sum / 2;}
    int main() {
    int V = 4;
    int D[] = {2, 2, 1, 1};
    printf("Edges: %d\n", count_undirected_edges(V, D));
    return 0;
    }
```

Introduction to Undirected Graph Search:

- Undirected Graph:

A collection of vertices (nodes) connected by edges without direction.

Example: If A and B are connected, movement is possible both ways (A ↔ B).

- Graph Search:

The process of visiting all vertices and edges in a graph in a systematic order.

- Purpose:

To find connections, detect paths, or explore the structure of the graph.

Need for Graph Search:

Graph search algorithms are used in many real-world areas:

🌐 Social Networks: Finding mutual friends or connections

🗺️ Maps & Navigation: Finding shortest routes between locations

💻 Network Analysis: Checking connected devices or data flow

🧠 Artificial Intelligence: Pathfinding in games and simulations

Depth First Search (DFS):

Concept:

DFS explores a graph deeply before moving to another branch.It uses a Stack (or recursion) to keep track of visited nodes.

- Steps:1. Start from a vertex and mark it as visited.

2. Visit an unvisited adjacent vertex.

3. Repeat until no unvisited vertex remains.4. Backtrack to explore remaining paths.

- Applications:

Detecting cycles in a graphFinding connected componentsPathfinding in mazes and puzzles

Breadth First Search (BFS):

Concept:

BFS explores the graph level by level. It uses a Queue to visit all neighbors first, then moves to the next level.

Uses:

Finding the shortest path in unweighted graphs.

Network broadcasting.

Exploring all reachable nodes from a source.

Example: For a graph

A — B

| |

C — D

Traversal order: A → B → C → D

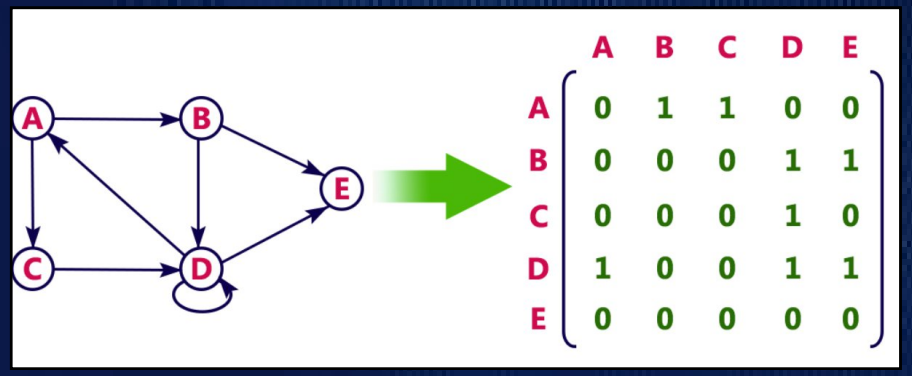
Counting Edges In A Directed Graph:

- A directed graph (or digraph) consists of a set of vertices **V** and a set of edges **E**, where each edge has a **direction** from one vertex to another.
  - G=(V,E)
  - V= set of vertices
  - E = set of directed edges
  - **Directed Edge:** Represented as an ordered pair (u,v) where u → v
- Maximum number of edges in a simple directed graph (no self-loops) = n(n−1)
- Maximum number of edges if self-loops allowed = n^2

Representation Of Directed Graphs:

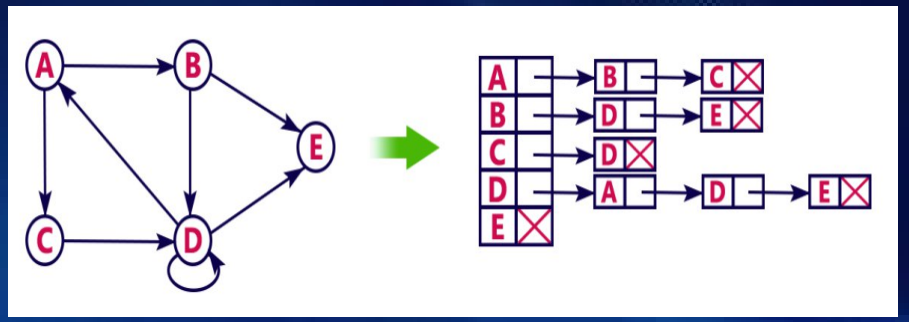
Adjacency Matrix:

- M[i][j]=1M[if edge i→j exists, else 0
- Number of edges = count of 1's in the matrix



Adjacency List:

- Each vertex stores a list of vertices it points to.
- Number of edges = sum of lengths of all adjacency lists.



Algorithm to Count Edges in a Directed Graph (Adjacency Matrix):

- **Input:** Number of vertices n and the adjacency matrix adj[n][n].
- Initialize count = 0.
- **For** each vertex i from 0 to n-1:
  - a. **For** each vertex j from 0 to n-1:
  - b. If adj[i][j] == 1, increment count by 1.
- **Output:**
- count as the total number of edges in the
- directed graph.

```
for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        if(adj[i][j] == 1)
            count++;
    }
}
```

Algorithm to Count Edges in a Directed Graph (Adjacency List):

- **Input:** Number of vertices n, number of edges e, and the adjacency list for each vertex.
- Initialize count = 0.
- **For** each vertex i from 0 to n-1:
    a. Set temp to the head of the adjacency list of vertex i.
    b. **While** temp is not NULL:
        i. Increment count by 1.
        ii. Move temp to the next node in the list.
- **Output:** count as the total number of edges in the directed graph.

```
for(i = 0; i < n; i++) {
    struct Node* temp = adj[i];
    while(temp != NULL) {
        count++;
        temp = temp->next;
    }
}
```

Application of Directed Graphs :

- Used to represent situations where relationships have **direction**.
- Common in systems where one entity affects or connects to another **one-way**.

Helps in analyzing network structure, flow and connectivity

Where Directed Graphs Are Used in Real Life:

- **Social Media Networks** – follow or connection systems
- **Airports & Flight Routes** – flights from one city to another
- **Google Maps / Navigation** – one-way roads and path directions
- **Project Management Tools** – task dependencies
- **Web Links & Search Engines** – how web pages link to each other

Directed Graph Search:

A directed graph (digraph) is a graph where edges have direction, meaning a connection from A → B is not the same as B → A.
Graph search helps us explore all reachable vertices following the direction of edges.

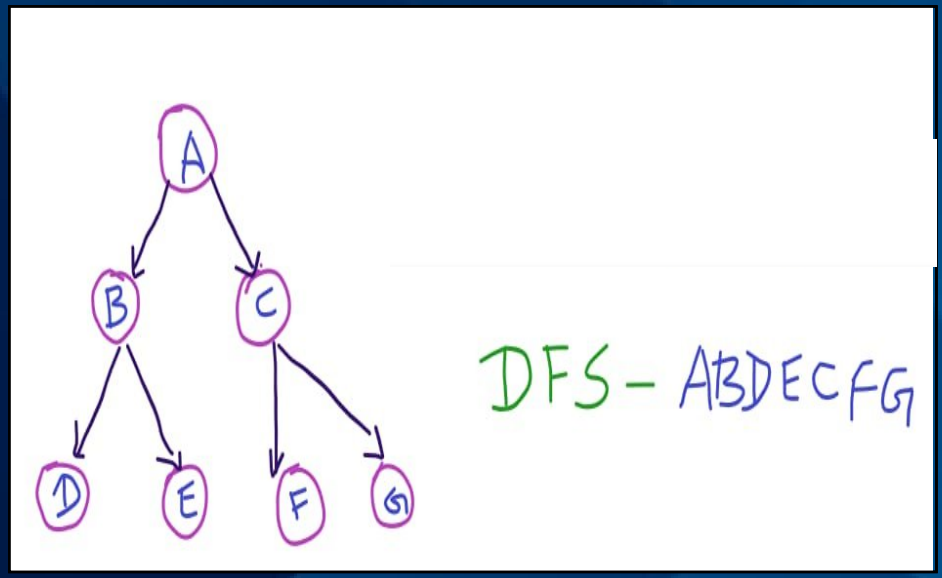
Need for Directed Graph Search:

- Directed graph search algorithms are useful in many fields:
- Task scheduling (dependencies between jobs),
- Navigation systems (one-way roads)

Types of Search Techniques:

Depth First Search (DFS):
DFS explores as far as possible along a directed path before backtracking.
Algorithm:

1. Start from a chosen vertex.
2. Mark it as visited.
3. Move to an unvisited vertex connected by a directed edge.
4. Repeat until no unvisited vertex remains.
5. Backtrack to explore other unvisited vertices.



Applications Of DFS Search:

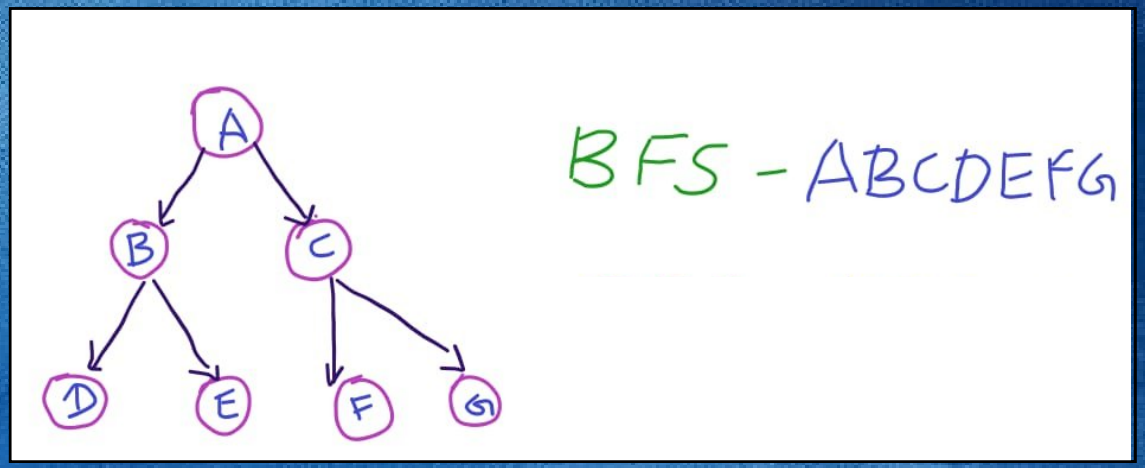
1.Detecting cycles in directed graphs.
2.Topological sorting of tasks.

Breadth First Search (BFS):

BFS visits all vertices level by level, following the direction of edges.
It uses a queue to process neighboring vertices before moving deeper.

Algorithm:

1.Start from a chosen vertex.
2.Mark it as visited and add it to the queue.
3.Dequeue a vertex and visit all its unvisited neighbors.
4.Enqueue the unvisited neighbors.
5.Repeat until the queue is empty.



Applications:

- Finding the shortest path in a directed unweighted graph.
- Web crawlers following page links.

THANKYOU